

WYDZIAŁ INFORMATYKI



Ćwiczenie laboratoryjne

Elementy graficznych interfejsów użytkownika w systemach mikroprocesorowych

Nazwa kodowa: **lcd**. Wersja **20251204**

Ada Brzoza-Zajęcka

abrzoza@agh.edu.pl

1 Wstęp

1.1 Cel ćwiczenia

Celem ćwiczenia jest zapoznanie się możliwościami tworzenia oprogramowania dla graficznych interfejsów użytkownika w oparciu o popularne narzędzia sprzętowe i programowe dla platform o ograniczonych zasobach sprzętowych.

1.2 Stanowisko testowe i plan pracy

Ćwiczenie przeprowadzone jest z wykorzystaniem platformy sprzętowej **Raspberry Pi Pico W** z mikrokontrolerem z RP2040 z rdzeniem ARM, bardzo popularnym modulem graficznego wyświetlacza LCD o rozdzielczości 320x240 pikseli wyposażonym w układ scalony sterownika ILI9341.

Od strony programowej użyjemy zintegrowanego środowiska Arduino (zalecana wersja to **2.3.5**) wraz z zainstalowanym wsparciem dla Raspberry Pi Pico:

<https://github.com/earlephilhower/arduino-pico>

a także z następującymi bibliotekami:

- Adafruit_GFX - bardzo popularna i prosta w obsłudze biblioteka graficzna dla systemów wbudowanych i urządzeń IoT o ograniczonych zasobach sprzętowych,
- Adafruit_ILI9341 - sterownik wyświetlacza LCD TFT ze układem scalonym ILI9341,
- ew innymi zależnie od potrzeb i kreatywności osób realizujących ćwiczenie.

Do obserwacji przebiegów użyjemy Digilent WaveForms.

1.3 Materiały pomocnicze

Do wykonania ćwiczenia niezbędne lub przydatne są następujące materiały:

- dokumentacja narzędzi wspierających rozwój oprogramowania dla Raspberry Pi Pico w środowisku Arduino:

<https://arduino-pico.readthedocs.io/en/latest/>

- w tym dokumentacja obsługi interfejsu SPI:

<https://arduino-pico.readthedocs.io/en/latest/spi.html> ,

- dokumentacja Adafruit GFX:

<https://github.com/adafruit/Adafruit-GFX-Library> ,

- dokumentacja biblioteki obsługi wyświetlacza Adafruit ILI9341:

https://github.com/adafruit/Adafruit_ILI9341 .

- kody źródłowe plików:

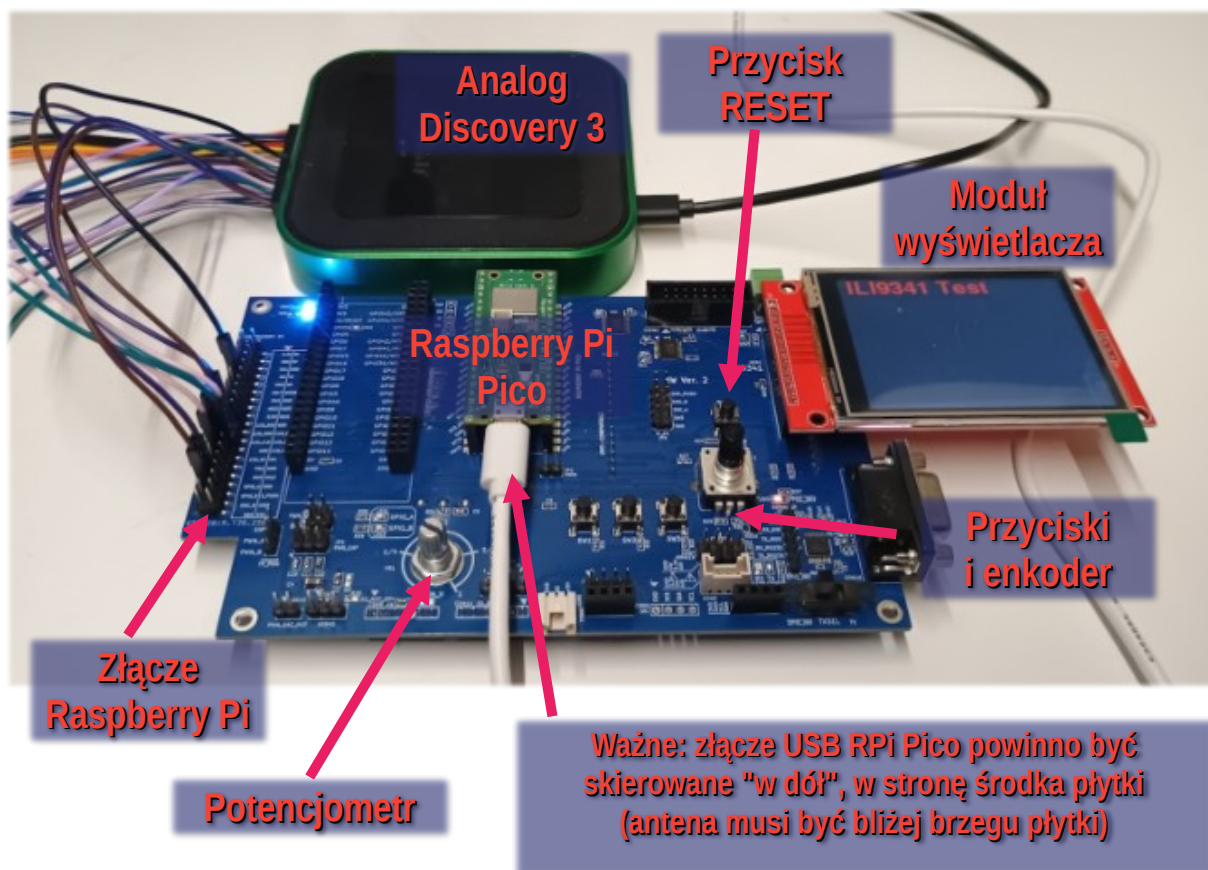
- \$HOME/Arduino/libraries/Adafruit_GFX_Library/Adafruit_GFX.cpp / .h
- \$HOME/Arduino/libraries/Adafruit_GFX_Library/Adafruit_SPITFT.cpp / .h
- \$HOME/Arduino/libraries/Adafruit_ILI9341/Adafruit_ILI9341.cpp / .h

2 Przygotowanie

Ćwiczenie wykonywane jest na unikalnej płytce testowej. Potrzebne fragmenty dokumentacji będą udostępniane na platformie UPEL. Do wykonywania pomiarów użyjemy uniwersalnego modułu Analog Discovery 3 (AD3) produkcji Digilent.

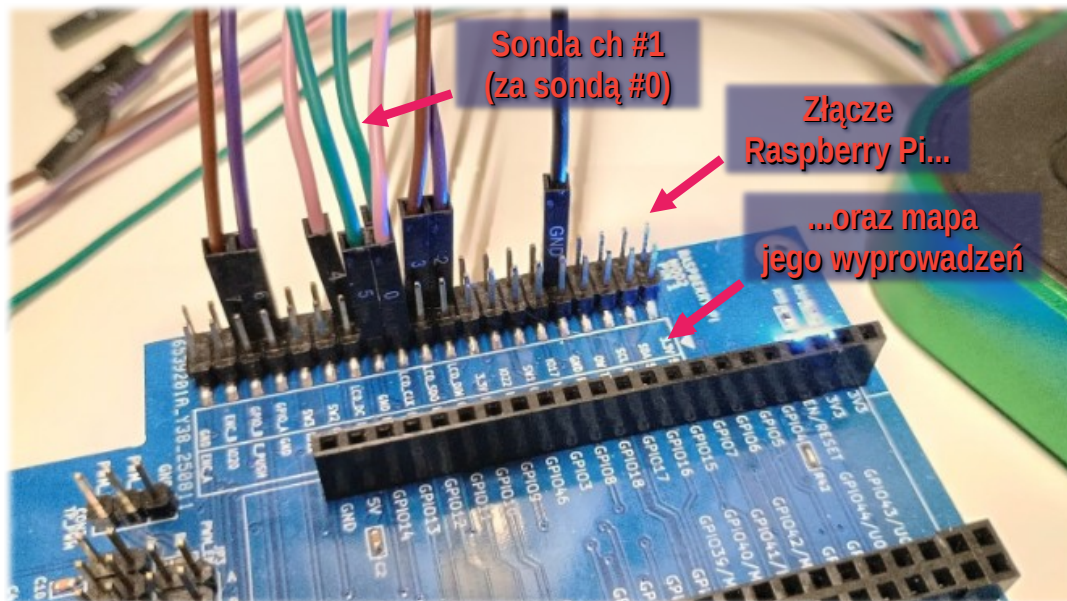
2.1 Elementy płytki testowej

Raspberry Pi Pico i wyświetlacz LCD ILI9341 powinny być podłączone do odpowiednio oznaczonych złączy na płytce, jak na rysunku poniżej↓



2.2 Podłączenie sond AD3

Sondy analizatora stanów logicznych najwygodniej może być podłączyć do złącza dla Raspberry Pi (aplikacyjnego) znajdującego się przy bocznej krawędzi płytki testowej ↓



Złącze to jest elektrycznie połączone wieloma wspólnymi sygnałami razem z innymi złączami na płytce, także ze złączem dla modułów ESP32-S3 oraz złączem dla modułów kompatybilnych z Arduino Uno. Interesujące nas teraz sygnały dla wyświetlacza LCD również pojawiają się na wszystkich wymienionych złączach. Zatem nie ma różnicy, do którego złącza dołączymy sondy analizatora - akurat złącze Raspberry Pi (aplikacyjnego 3, 4, 5, Zero itp.) znajduje się przy krawędzi płytki i jest po prostu najwygodniejsze przy pomiarach w niniejszym ćwiczeniu, choć samego komputera Raspberry Pi (aplikacyjnego) użyjemy dopiero w innych ćwiczeniach.

Odwzorowanie wyprowadzeń i sposób podłączenia sond są przedstawione poniżej.

Uwaga: w różnych modułach sygnały wejściowe i wyjściowe SPI są różnie nazywane. Dla kompatybilności w wielu miejscach wciąż można spotkać nierekomendowane obecnie nazwy MISO i MOSI, od których jednak stopniowo odchodzi się (rozwijając ich akronimy dość łatwo można przekonać się dlaczego).

W naszym przypadku sygnały:

MOSI (od master output slave input) w AD3 = DIN (od data input) w LCD

MISO (od master input slave output) w AD3 = SDO (od serial data output) w LCD

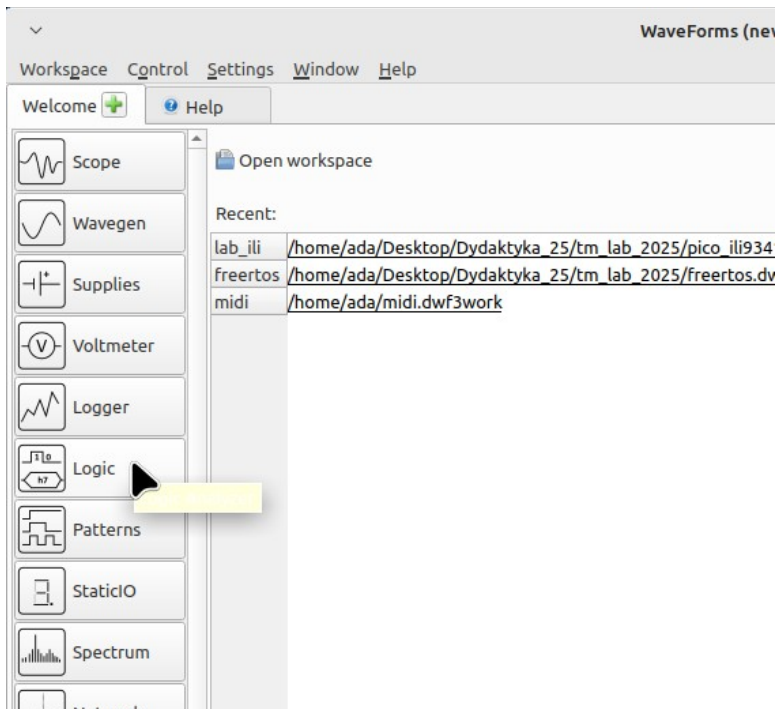
Złącze dla komputerów Raspberry Pi (aplikacyjnych) -- używane wyłącznie jako interfejs do podłączenia sond w ćwiczeniu					
S#	← Numery sond →				S#
	RPI				
	3V3	1	2	5V	
	SDA	3	4	5V	
	SCL	5	6	GND	
	IO4	7	8	TX	
G	GND	9	10	RX	
	IO17	11	12	WS2812B	
	SW1	13	14	GND	
	IO22	15	16	ONEWIRE	
	3V3	17	18	IO24	
#2	LCD_DIN	19	20	GND	
#3	LCD_SDO	21	22	HUB_E	
#1	LCD_CLK	23	24	LCD_CS	#0
	GND	25	26	LCD_RST	#5
#4	LCD_DC	27	28	IO1	
	SW2	29	30	GND	
	SW3	31	32	ENC_PUSH	
#6	GPIO_A	33	34	GND	
#7	GPIO_B	35	36	IO16	
	ENC_B	37	38	ENC_A	
	GND	39	40	IO21	

Złącze dla modułu Raspberry Pi Pico używanego w ćwiczeniu					
RPI Pico					
LCD_SDO	GP0	1	40	VBUS	
SW2	GP1	2	39	VSYS	
	GND	3	38	GND	
LCD_CLK	GP2	4	37	3V3_EN	
LCD_DIN	GP3	5	36	3V3	
SW3	GP4	6	35	VREF	
LCD_DC	GP5	7	34	GP28_A2	WS2812B
	GND	8	33	AGND	
LCD_RST	GP6	9	32	GP27_A1	AIN
GPIO_A	GP7	10	31	GP26_A0	ONEWIRE
LCD_CS	GP8	11	30	RUN	
GPIO_B	GP9	12	29	GP22	SW1
	GND	13	28	GND	
LCD_BL	GP10	14	27	GP21	RX
	GP11	15	26	GP20	TX
ENC_A	GP12	16	25	GP19	SD_MOSI
ENC_B	GP13	17	24	GP18	SD_CLK
	GND	18	23	GND	
SDA	GP14	19	22	GP17	SD_CS/ GPIO_INV
SCL	GP15	20	21	GP16	SD_MISO

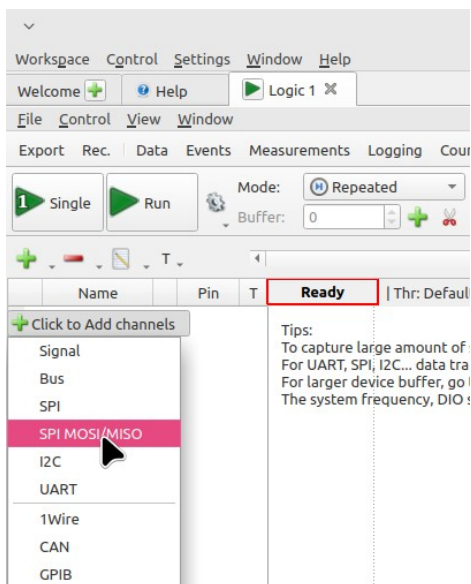
2.3 Konfiguracja WaveForms

Mając przygotowany AD3 i podłączone sondy otwieramy WaveForms.

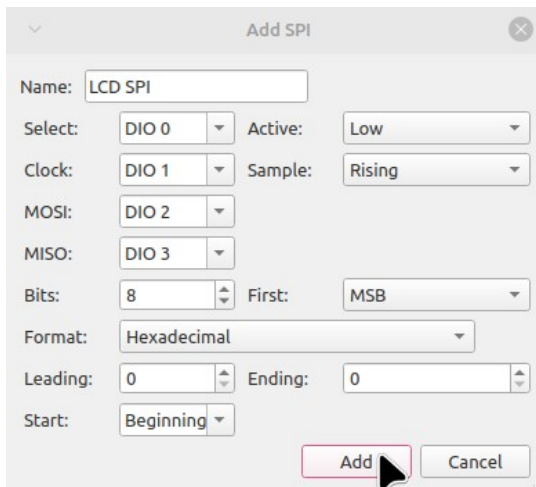
Wybieramy aplikację Logic ↓



Logic otworzy nam się w nowej zakładce. Dodajemy kanały pomiarowe wybierając **Click to Add channels** → **SPI MOSI/MISO** ↓



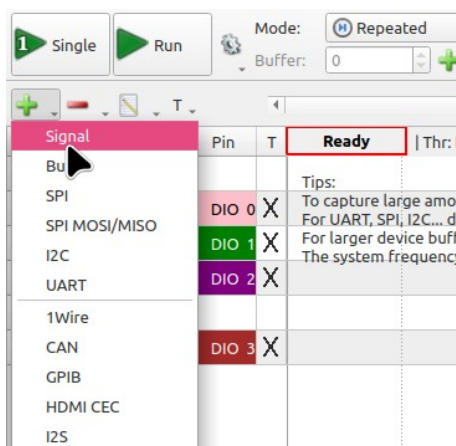
Prawdopodobnie nie będzie trzeba dużo zmieniać w konfiguracji - sprawdzimy jednak czy wszystko się zgadza. Można także nadać tym sygnałom adekwatną nazwę, np. LCD SPI. Zatwierdzamy przyciskiem **Add**.



Pamiętamy, że:

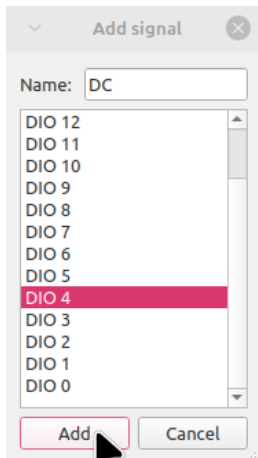
- MOSI (*master output slave input*) w AD3 = DIN (*data input*) w LCD
(kierunek: **Pico** → **LCD**)
- MISO (*master input slave output*) w AD3 = SDO (*serial data output*) w LCD
(kierunek: **Pico** ← **LCD**)

Dodajemy kolejne sygnały klikając **+** → **Signal**:



Wybierając numer sondy (*DIO*n) i wpisując nazwy (w polu *Name* jak poniżej) dodajemy kolejne sygnały:

DIO4 → DC



a także...

DIO5 → RST

DIO6 → GPIO_A

DIO7 → GPIO_B

(nazwy wpisujemy jedynie dla wygody i przejrzystości - nie mają one znaczenia dla obserwacji poza tym, że zmniejszają prawdopodobieństwo pomylenia sygnałów).

3 Oprogramowanie

W ramach sprawozdania z realizacji ćwiczenia, należy zaprezentować osobie prowadzącej uzyskane efekty realizacji ćwiczeń, a także efekty poszczególnych etapów umieszczać na platformie UPEL wraz z zrzutami ekranu prezentującymi działanie.

Część zadań do wykonania w tym ćwiczeniu polega na świadomej analizie dostępnego kodu oraz na przeprowadzeniu własnych eksperymentów.

Układy peryferyjne dołączone są do Raspberry Pi Pico w sposób przedstawiony w tabeli poniżej. Może to być przydatne przy tworzeniu oprogramowania w dalszej części ćwiczenia.

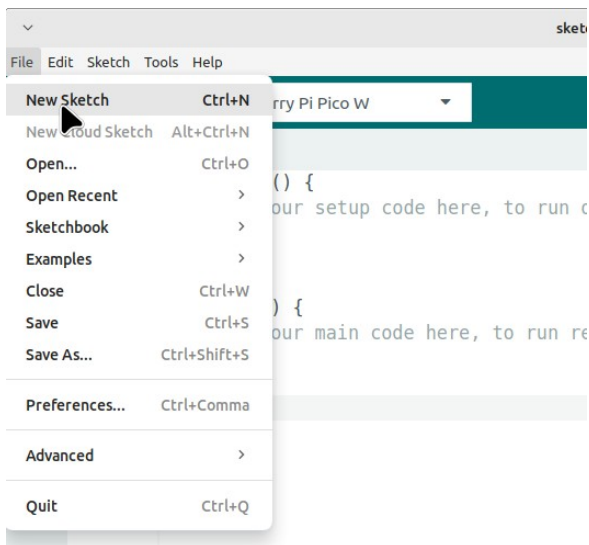
RPi Pico					
LCD_SDO	GP0	1	40	VBUS	
SW2	GP1	2	39	VSYS	
	GND	3	38	GND	
LCD_CLK	GP2	4	37	3V3_EN	
LCD_DIN	GP3	5	36	3V3	
SW3	GP4	6	35	VREF	
LCD_DC	GP5	7	34	GP28_A2	WS2812B
	GND	8	33	AGND	
LCD_RST	GP6	9	32	GP27_A1	AIN
GPIO_A	GP7	10	31	GP26_A0	ONEWIRE
LCD_CS	GP8	11	30	RUN	
GPIO_B	GP9	12	29	GP22	SW1
	GND	13	28	GND	
LCD_BL	GP10	14	27	GP21	RX
	GP11	15	26	GP20	TX
ENC_A	GP12	16	25	GP19	SD_MOSI
ENC_B	GP13	17	24	GP18	SD_CLK
	GND	18	23	GND	
SDA	GP14	19	22	GP17	SD_CS/ GPIO_INV
SCL	GP15	20	21	GP16	SD_MISO

3.1 Obsługa Arduino i interakcja z RPi Pico

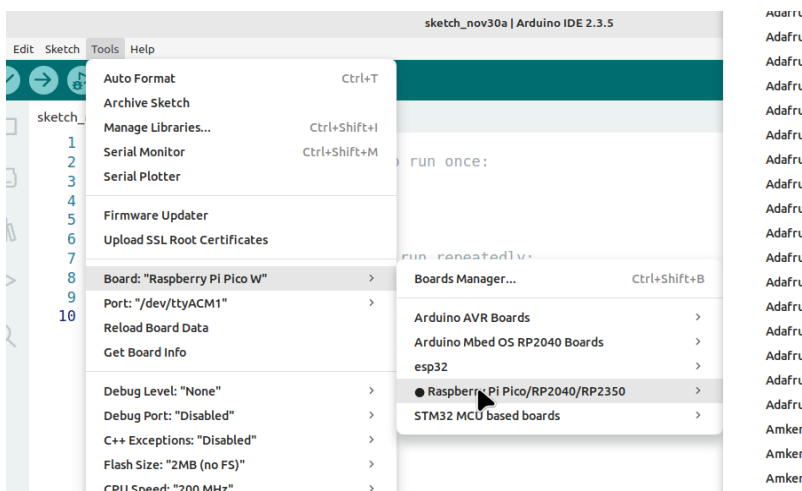
Środowisko Arduino znajdziemy w katalogu `$HOME/tools` w formie AppImage. Jeśli w katalogu znajduje się więcej wersji AppImage tego środowiska, to zalecane jest uruchomienie 2.3.5 lub innej najnowszej dostępnej.

3.1.1 Konfiguracja projektu

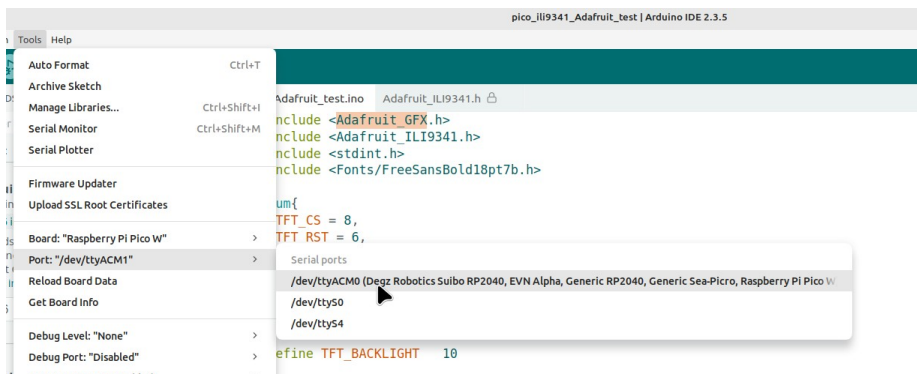
W środowisku Arduino tworzymy nowy projekt klikając **File** → **New Sketch**.



Wybieramy z menu **Tools** → **Board: Raspberry Pi Pico/RP2040/RP2350** → **Raspberry Pi Pico W**.

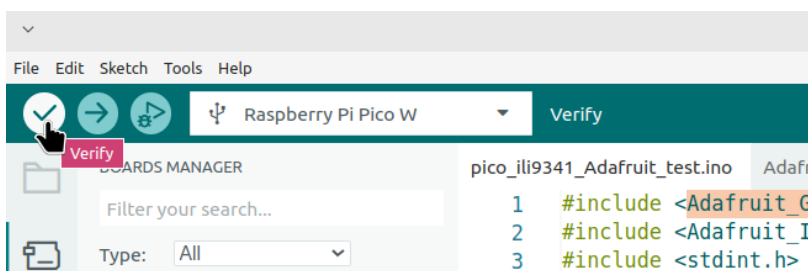


Zależnie od tego, dla jakiej platformy było wcześniej używane środowisko Arduino, to będzie można **na tym etapie lub później wybrać port szeregowy**, przez który będziemy komunikować się z Raspberry Pi Pico. Port będzie dostępny, gdy Raspberry Pi Pico będzie miało możliwość komunikacji przez ten interfejs. Port powinien być widoczny jako `/dev/ttyACMx`, tj. `/dev/ttyACM0`, `1`, `2`, Ogólnie celujemy w coś wyglądającego jak port poniżej - pamiętajmy, że może to być jakaś dość długa nazwa z całą listą platform ↓

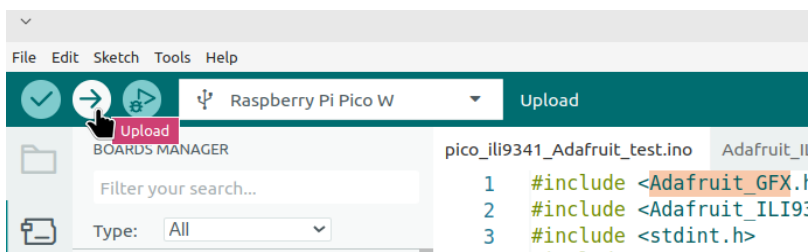


3.1.2 Kompilacja i programowanie

Aby skompilować projekt w Arduino należy kliknąć ikonkę "checkmarka" **Verify**:



Programowanie przeprowadzimy wciskając przycisk strzałki **Upload**:



Ale najpierw...

...**ale najpierw** może być potrzebne uruchomienie RPi Pico w trybie wewnętrznego bootloadera pozwalającego na programowanie przez Arduino. Aby mieć pewność, że Raspberry Pi Pico jest w trybie gotowości do programowania z Arduino i zgłasza się jako pamięć masowa:

1. Wciskamy i trzymamy mały biały przycisk **BOOTSEL** na płytce Raspberry Pi Pico
2. Trzymając przycisk BOOTSEL wykonujemy zerowanie płytki (najwygodniej przyciskiem

RESET na płytce testowej)

3. Zwalniamy **najpierw RESET**, a **później BOOTSEL**.

Powyższą procedurę możemy próbować zastosować także później, jako jedną z opcji "awaryjnych", gdy z różnych powodów nie będzie się dało zaprogramować Raspberry Pi Pico, np. w przypadku komunikatu *No drive to deploy*.

```
Converting to uf2, output size: 715776, start address: 0x2000
Scanning for RP2040 devices
No drive to deploy.
Failed uploading: uploading error: exit status 1
```

3.2 Tworzenie projektu startowego

Niezależnie czy tworzymy oprogramowanie w VSCode czy w Arduino, to jeśli używamy frameworka Arduino, kod będzie podzielony na dwie główne sekcje w funkcjach `setup` i `loop`:

```
void setup() {
// put your setup code here, to run once:

}

void loop() {
// put your main code here, to run repeatedly:

}
```

Jak widać w komentarzach: `setup` wykonuje się raz przy starcie programu, a `loop` jest wywoływana cyklicznie. W przypadku mikrokontrolerów wielordzeniowych, oprogramowanie może wspierać więcej niż jedną funkcję `loop`, oczywiście o nieco innej nazwie.

Zacznijmy od dodania przydatnych nagłówków na samym początku pliku skrepcu:

```
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
```

Następnie przydadzą się definicje wyprowadzeń GPIO platformy Raspberry Pi Pico, do których podłączone są wyprowadzenia wyświetlacza. Z racji, że w Arduino wyprowadzenia GPIO mają przypisane liczby naturalne, to można użyć zarówno stałych `int`, typu wyliczeniowego `enum`, a nawet oldschoolowych makrodefinicji `#define`. Poniżej przykładowy miks podejść:

```
enum{
TFT_CS = 8,
TFT_RST = 6,
TFT_DC = 5,
TFT_MOSI = 3,
TFT_CLK = 2,
TFT_MISO = 0
};
```

```
#define TFT_BACKLIGHT 10
```

Następnie jako zmienną globalną możemy użyć wskaźnik do obiektu, który będzie reprezentował sterownik wyświetlacza, np.

```
Adafruit_ILI9341* display = nullptr;
```

Obiekt ten koniecznie trzeba zainicjalizować w funkcji setup, jednak zanim to zrobimy, w setup warto dopisać parę przydatnych linii kodu:

```
//inicjalizujemy komunikację przez port szeregowy ttyACMx
```

```
Serial.begin(115200);
```

```
//dzięki temu zmniejszymy szansę "zgubienia" pierwszych komunikatów
```

```
delay(1000);
```

```
//konfigurujemy jako wyjście pin sterujący włączaniem podświetlenia wyświetlacza
```

```
pinMode(TFT_BACKLIGHT, OUTPUT);
```

```
//włączamy podświetlenie
```

```
digitalWrite(TFT_BACKLIGHT, HIGH);
```

```
//Komunikat "żyję"
```

```
Serial.printf("ILI9341 Display Test\n");
```

```
//wreszcie inicjalizacja obiektu reprezentującego wyświetlacz
```

```
display = new Adafruit_ILI9341(TFT_CS, TFT_DC, TFT_MOSI, TFT_CLK, TFT_RST, TFT_MISO);
```

```
//...oraz właściwa inicjalizacja sterownika wyświetlacza
```

```
display->begin();
```

```
//próba odczytu danych z wyświetlacza:
```

```
uint8_t rd = display->readcommand8(ILI9341_RDMODE);
```

```
Serial.printf("Response to RDMODE: 0x%02X",rd);
```

```
//przygotowanie i wypisanie tekstu
```

```
display->fillScreen(ILI9341_BLACK); //czyszczenie całego ekranu (na czarno;))
```

```
display->setRotation(3); //orientacja/obrót obrazu pasująca do jego fizycznego ułożenia
```

```
display->setFont(NULL); //czcionka wbudowana (na razie)
```

```
display->setTextSize(3); //wybór rozmiaru tekstu
```

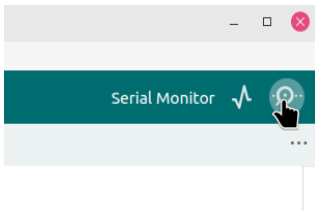
```
display->setTextColor(ILI9341_RED); //kolor tekstu
```

```
display->setCursor(10, 30); //ustawienie kursora (w pikselach)
```

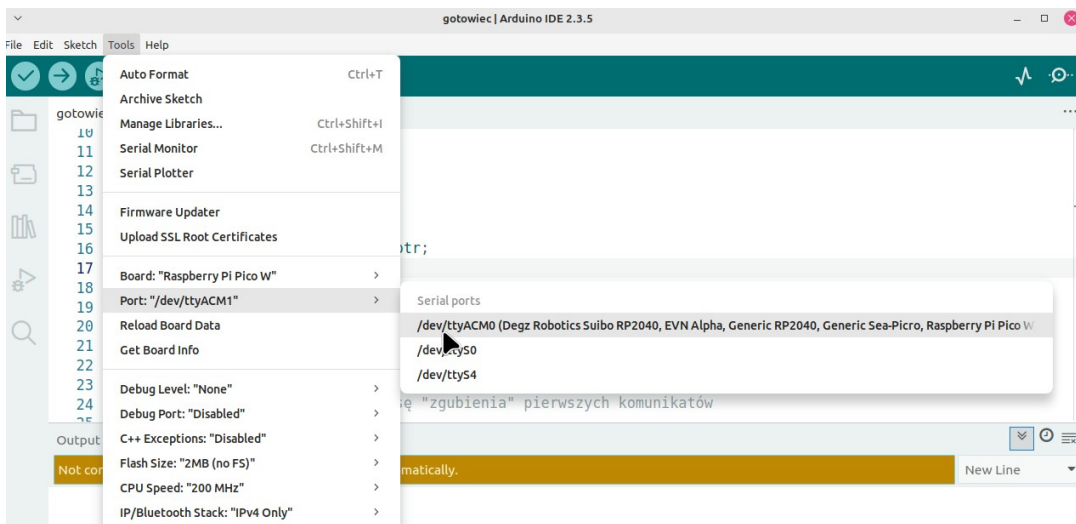
```
display->printf("ILI9341 Test\n"); //wypisanie komunikatu na wyświetlaczu
```

3.2.1 Uwagi i spostrzeżenia

Aby zobaczyć komunikaty wyświetlane przez port szeregowy, to w Arduino bardzo wygodny jest mechanizm monitora portu szeregowego. Uruchamiamy go klikając ikonkę:



Jeśli nie możemy się połączyć, to wybieramy port (jak w poprzednim punkcie instrukcji):



Powinniśmy otrzymać z programu komunikat mniej-więcej taki:



Zauważmy także, że czas odświeżania wyświetlacza, który możemy obserwować przy starcie programu (po wciśnięciu RESET) jest co najmniej "niedzisiejszy", ponieważ w aktualnej konfiguracji sterownik wyświetlacza działa w trybie całkowicie programowym, używając pinów GPIO do implementacji interfejsu SPI - zmienimy to później, ale najpierw wykonamy parę obserwacji samego interfejsu SPI (w aktualnej, wstępnej konfiguracji jest to łatwiejsze ze względu na niższe częstotliwości).

Ponadto czcionka na wyświetlaczu także wygląda raczej jak w latach '80 XX wieku. Tym także

zajmiemy się za chwilę.

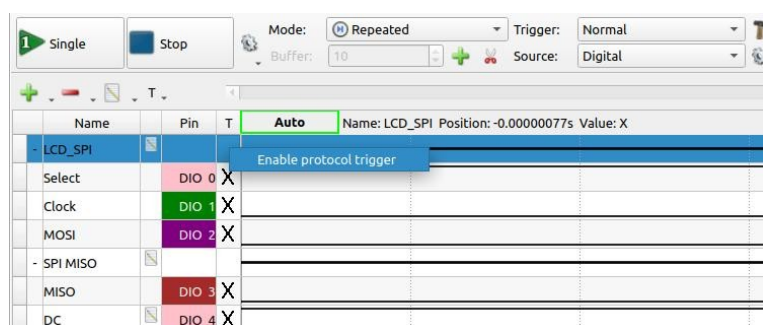
Pomimo wyżej wymienionych mankamentów tak czy inaczej dokumentujemy aktualny etap prac: robimy screenshoty, zdjęcia z działania urządzenia, kopiujemy kod projektu i umieszczamy wszystko na UPEL.

3.3 Obserwacje SPI

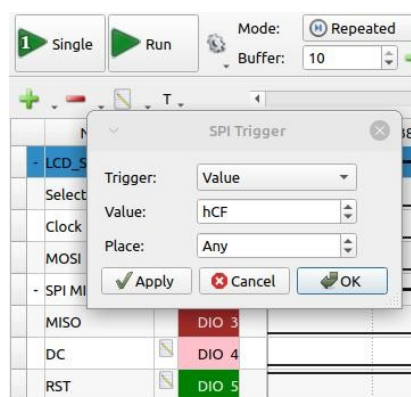
Spróbujmy zarejestrować początek inicjalizacji wyświetlacza LCD. Przy okazji popatrzymy na rzeczywiste przebiegi interfejsu SPI i poznamy szczegóły troszkę bardziej zaawansowanej obsługi WaveForms i AD3 (*protocol trigger*).

Często z racji bardzo obszernej dokumentacji i wysokiego poziomu złożoności układów scalonych sterowników wyświetlaczy, ich inicjalizacja bardziej przypomina "czarną magię" niż pracę inżynierów - w praktyce sprowadza się ona często do wysłania odpowiedniej sekwencji poleceń wg przykładu producenta lub - jeszcze gorzej - kompilacji paru przykładów znalezionych w internecie. W przypadku ILI9341 również mamy do czynienia z taką sytuacją. Dla tych, którzy nie wierzą w te słowa, kopia dokumentacji sterownika wyświetlacza znajduje się na UPEL :P

Zacznijmy obserwację. Aby w WaveForms dodać wyzwalanie rejestracji konkretną liczbą pojawiającą się na magistrali, należy dodać tzw. **protocol trigger** czyli coś, co sprawi, że rejestracja rozpocznie się od wykrycia bardzo konkretnego stanu. Klikamy zatem na pustym polu **T** przy nagłówku LCD SPI i wybieramy **Enable protocol trigger**.

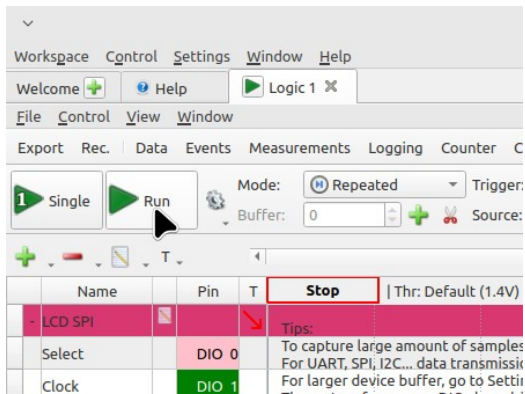


Następnie jeszcze raz klikamy na strzałkę w dół, która się pojawi i ustawiamy wyzwalanie na liczbę 0xCF

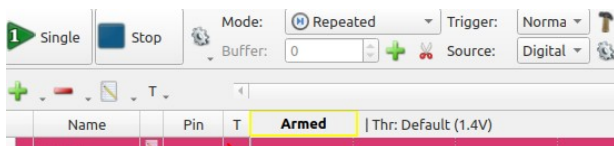


Zatwierdzamy OK.

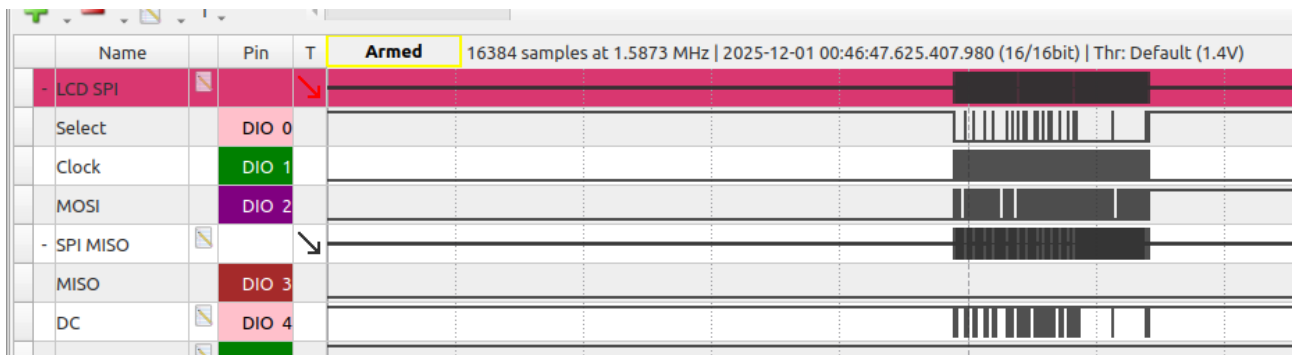
Następnie klikamy Run



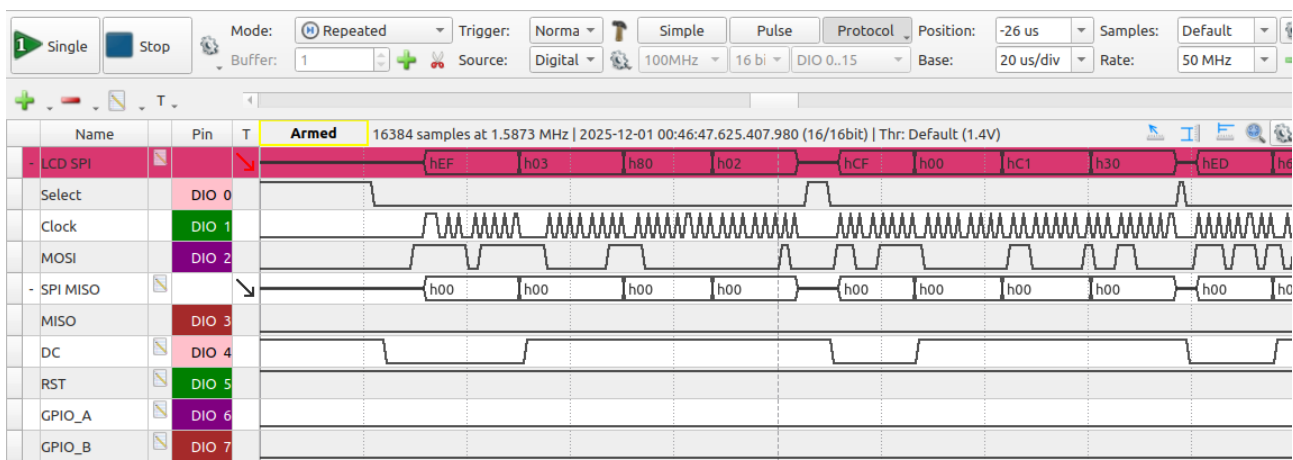
przez co uruchamiamy oczekiwanie na trigger (stan Armed)...



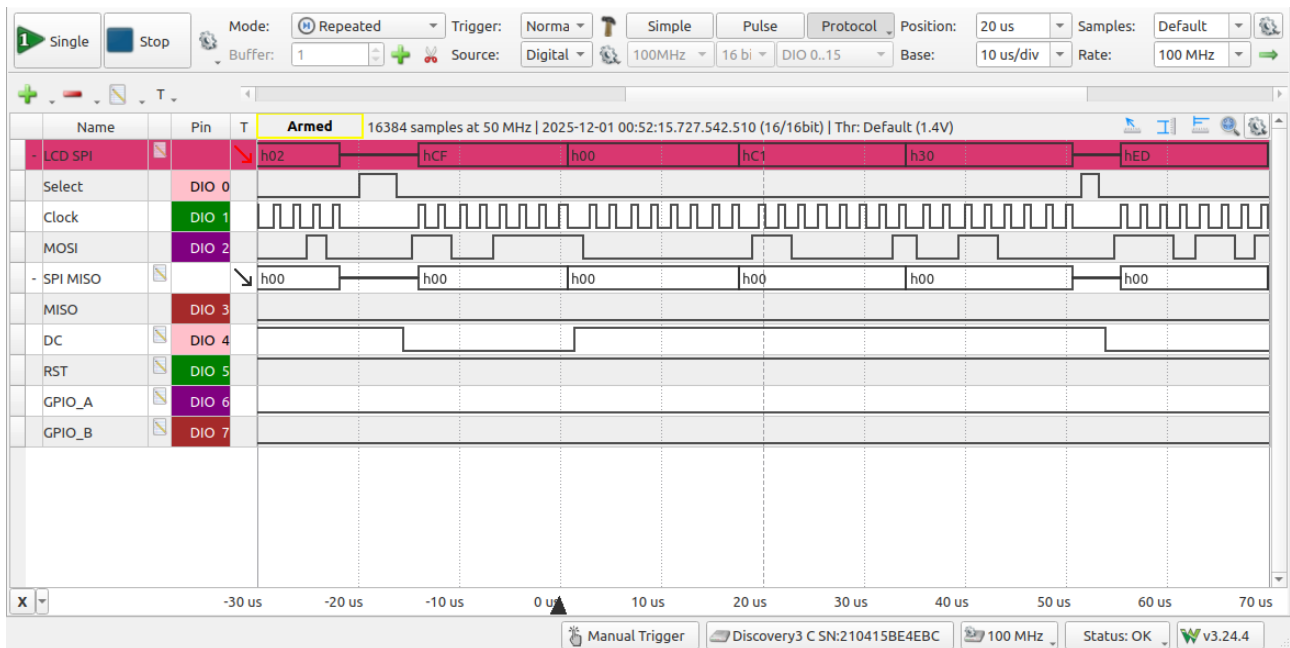
Gdy wykonamy zerowanie płytki przyciskiem RESET, to po chwili prawdopodobnie zobaczymy pierwsze przebiegi. Mogą jednak nie być ani piękne, ani przejrzyste, np.:



Gdy powiększymy przebiegi (np. rolką myszy), to może się okazać także, że są one zarejestrowane ze zbyt niską częstotliwością próbkowania (jak poniżej, widać niewłaściwy wygląd sygnału zegarowego, te "szpilki").



Dlatego, mając ustawiony "zoom" na takim poziomie, że rzeczywiście coś dostrzegamy możemy spróbować ponownie zarejestrować przebiegi (ponownie **Run**, **RESET**). Tym razem powinny być dokładniejsze, np.:

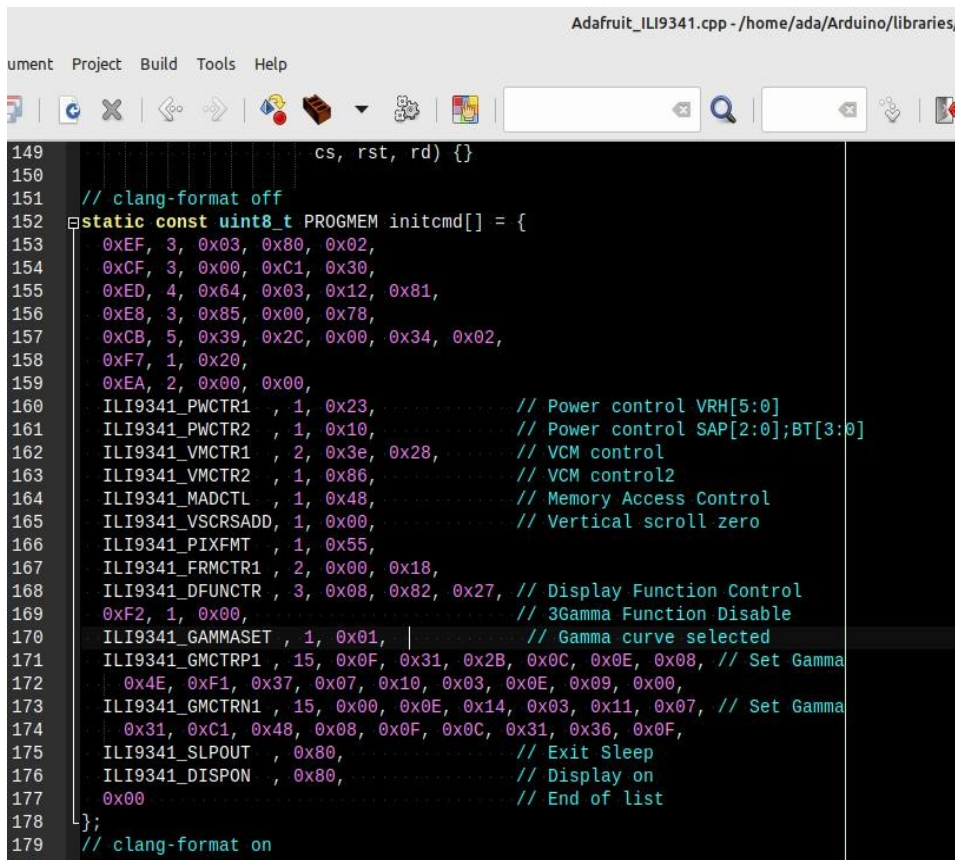


Gdy uznamy zarejestrowane przebiegi za odpowiednio przejrzyste, możemy wcisnąć przycisk Stop, zrobić zrzut ekranu i umieścić go na UPEL. Przebiegów jednak nie usuwamy - za chwilę wykonamy jeszcze ich pomiary.

Gdy zajrzemy do pliku

`$HOME/Arduino/libraries/Adafruit_ILI9341/Adafruit_ILI9341.cpp`

To możemy znaleźć całą tablicę zawierającą komendy inicjalizujące wyświetlacz (`initcmd`):



```
149         cs, rst, rd) {}
150
151 // clang-format off
152 static const uint8_t PROGMEM initcmd[] = {
153     0xEF, 3, 0x03, 0x80, 0x02,
154     0xCF, 3, 0x00, 0xC1, 0x30,
155     0xED, 4, 0x64, 0x03, 0x12, 0x81,
156     0xE8, 3, 0x85, 0x00, 0x78,
157     0xCB, 5, 0x39, 0x2C, 0x00, 0x34, 0x02,
158     0xF7, 1, 0x20,
159     0xEA, 2, 0x00, 0x00,
160     ILI9341_PWCTR1, 1, 0x23, // Power control VRH[5:0]
161     ILI9341_PWCTR2, 1, 0x10, // Power control SAP[2:0];BT[3:0]
162     ILI9341_VMCTR1, 2, 0x3e, 0x28, // VCM control
163     ILI9341_VMCTR2, 1, 0x86, // VCM control2
164     ILI9341_MADCTL, 1, 0x48, // Memory Access Control
165     ILI9341_VSCRSADD, 1, 0x00, // Vertical scroll zero
166     ILI9341_PIXFMT, 1, 0x55,
167     ILI9341_FRMCTR1, 2, 0x00, 0x18,
168     ILI9341_DFUNCTR, 3, 0x08, 0x82, 0x27, // Display Function Control
169     0xF2, 1, 0x00, // 3Gamma Function Disable
170     ILI9341_GAMASET, 1, 0x01, // Gamma curve selected
171     ILI9341_GMCTRP1, 15, 0x0F, 0x31, 0x2B, 0x0C, 0x0E, 0x08, // Set Gamma
172     0x4E, 0xF1, 0x37, 0x07, 0x10, 0x03, 0x0E, 0x09, 0x00,
173     ILI9341_GMCTRN1, 15, 0x00, 0x0E, 0x14, 0x03, 0x11, 0x07, // Set Gamma
174     0x31, 0xC1, 0x48, 0x08, 0x0F, 0x0C, 0x31, 0x36, 0x0F,
175     ILI9341_SLP_OUT, 0x80, // Exit Sleep
176     ILI9341_DISP_ON, 0x80, // Display on
177     0x00 // End of list
178 };
179 // clang-format on
```

I, rzeczywiście, komenda `0xCF` jest drugą wydawaną dla wyświetlacza w procesie inicjalizacji. Dalej za nią widzimy informację, że do komendy będą 3 bajty argumentów, po czym następują właściwe argumenty. Biblioteka `Adafruit_ILI9341` jest jednym z przykładów "ładniej" napisanego kodu inicjalizacji wyświetlacza, a sam sterownik `ILI9341` jest od dawna znany, używany i dobrze udokumentowany. W sytuacjach ekstremalnych możemy się spotkać z ciągiem bajtów w tablicy, który należy wysłać do wyświetlacza "bo tak mówi producent" albo "bo ktoś to tak shackował".

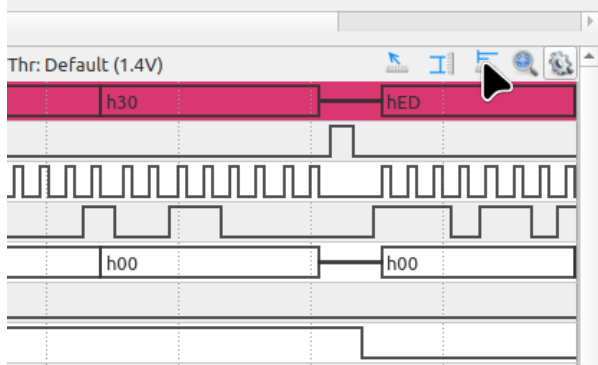
Dla dociekliwych fragment dokumentacji opisujący komendę 0xCF:

8.4.2 Power control B (CFh)

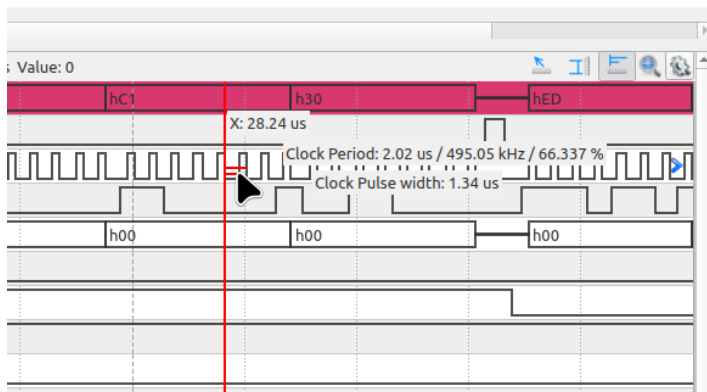
CFh	Power control B													
	D/CX	RDX	WRX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	HEX	
Command	0	1	↑	XX	1	1	0	0	1	1	1	1	CFh	
1 st Parameter	1	1	↑	XX	0	0	0	0	0	0	0	0	00	
2 nd Parameter	1	1	↑	XX	1	PCEQ	DRV_ena	Power control[1:0]			0	0	1	81
3 rd Parameter	1	1	↑	XX	DRV_vml[2:1]		1	DC_ena	DRV_vml[0]	DRV_vmh[2:0]		30		
Description	2 nd parameter: power control[1:0]													
	Only setting power control [1:0]=11, the VGH and VGL voltage level follow the table below.													
	BT [2:0]		DDVDH		VGH		VGL							
	0 0 0		VCI x 2		VCI x 7		-VCI x 4							
	0 0 1				-VCI x 3									
	0 1 0				-VCI x 4									
	0 1 1		VCI x 6		-VCI x 3									
	bit[5]: DRV_ena : For VCOM driving ability enhancement, DRV_ena = 1: Enable, and vice versa													
	bit[6]: PCEQ: PC and EQ operation for power saving													
	0:disable this function													
	1:enable this function													
	3 rd parameter: default: 30h													
bit[2:0]: DRV_vmh[2:0] 3'b000 adjust over drive width for VMH(000: 1 op_clk ~111: 8 op_clk)														
bit[3]: DRV_vml[0] 1'b0														
bit[4]: DC_ena: Discharge path enable. Enable high for ESD protection, 1: enable and vice versa														
bit[7:6]: DRV_vml[2:1] 2'b00														

3.4 Pomiar czasów na przebiegach

Zanim jednak wyczyścimy obszar rysowania przebiegów, warto zmierzyć aktualną (ewidentnie zbyt niską) częstotliwość sygnału zegarowego. Klikamy zatem na tę ikonkę:



I naprowadzamy kursor nad sygnał zegarowy. Sprawdzamy częstotliwość tego sygnału:



Prawdopodobnie będzie wynosiła ok. 500 kHz. To mało jak na możliwości tego wyświetlacza.

3.5 Sprzętowy SPI i szybszy zegar

Prostym sposobem przyspieszenia sygnału zegarowego jest zastosowanie wspomaganego sprzętowo interfejsu.

W przypadku SPI dla Pico w Arduino możemy to zrobić konfigurując obiekt SPI a następnie podając referencję do niego przy wywołaniu konstruktora klasy Adafruit_ILI9341.

Spróbujemy podmienić ten fragment kodu, a następnie sprawdzimy szybkość odświeżania i częstotliwość sygnału zegarowego SPI :)

```
//konfiguracja SPI wspomagane go sprz etowo - przed inicjalizacj  wy szwielacza
SPI.setRX(TFT_MISO);
SPI.setTX(TFT_MOSI);
SPI.setSCK(TFT_CLK);
SPI.begin();
//inicjalizacja obiektu reprezentuj ącego wy szwielacz ze sprz etowym SPI
display = new Adafruit_ILI9341(&SPI, TFT_DC, TFT_CS, TFT_RST);
```

3.6 Ładniejsze czcionki

Zajrzyjmy do katalogu:

\$HOME/Arduino/libraries/Adafruit_GFX_Library/Fonts/

ada	Arduino	libraries	Adafruit_GFX_Library	Fonts
 FreeMono9pt7b.h			 FreeSans9pt7b.h	 F
 FreeMono12pt7b.h			 FreeSans12pt7b.h	 F
 FreeMono18pt7b.h			 FreeSans18pt7b.h	 F
 FreeMono24pt7b.h			 FreeSans24pt7b.h	 F
 FreeMonoBold9pt7b.h			 FreeSansBold9pt7b.h	 F
 FreeMonoBold12pt7b.h			 FreeSansBold12pt7b.h	 F
 FreeMonoBold18pt7b.h			 FreeSansBold18pt7b.h	 F
 FreeMonoBold24pt7b.h			 FreeSansBold24pt7b.h	 F

Znajdziemy w nim duuużo plików nagłówkowych z czcionkami. Ale z nich skorzystać?

Wystarczy dodać nagłówek z wybraną czcionką, np.

```
#include <Fonts/FreeSansBold18pt7b.h>
```

A następnie w kodzie napisać zamiast

```
display->setFont(NULL); //czcionka wbudowana (na razie)
display->setTextSize(3); //wybór rozmiaru tekstu
```

informację o wybranej czcionce, np.

```
display->setFont(&FreeSansBold18pt7b);
```

Sprawdźmy czy zadziała. Jeśli tak, to - klasycznie - proszę udokumentować postępy, zaprezentować osobie prowadzącej i przechodzimy dalej :)

3.7 Praca samodzielna

Proszę utworzyć kod rysujący obrazek składający się z kilku figur geometrycznych wypełnionych kolorem. Jako minimum proszę narysować i napisać:

- jeden lub więcej wypełniony kolorem prostokąt,
- jeden lub więcej wypełniony kolorem trapez,
- koło.

3.8 Realizacja kreatywnego zadania

Używając biblioteki Adafruit_GFX należy napisać efektowny program wykorzystujący elementy interfejsu graficznego: wyświetlacz LCD i/lub interfejs sterujący wykorzystujący przyciski lub enkoder. W ramach realizacji zadania można napisać np.:

- prostą grę,
- wizualizację prostego, ale ciekawego algorytmu (np. The game of life),
- animację wg zadanych statycznie lub dynamicznie parametrów,
- wyświetlanie bitmap (odpowiednio skonwertowanych do kodu).

Mile widziane jest pozytywne zaskoczenie osoby prowadzącej laboratorium kreatywnością zespołu w realizacji tego zadania.

Z racji, że biblioteka Adafruit_GFX jest dobrze znana i udokumentowana, istnieje wiele opracowań na jej temat a także dodatkowych narzędzi, np. konwertery bitmap, czcionek.

Dla osób jeszcze bardziej zainteresowanych, można zastosować enkoder obrotowy i przyciski.

Odczyt przycisku realizujemy w następujący sposób:

Najpierw w setup wykonujemy prostą (często nieobowiązkową) inicjalizację:

```
pinMode(nr_pinu, INPUT_PULLUP); //tutaj z wewn. pull-up
```

A dalej właściwie w dowolnym miejscu programu możemy ten przycisk odczytać:

```
uint8_t stan_wyprowadzenia = digitalRead(BUTTON_PIN);
```

Obsługa enkodera jest nieco bardziej skomplikowana. Jednak hint: można spróbować zastosować odpowiednią bibliotekę, np. EEncoder:

<https://github.com/brianvarren/EEncoder>

4 Zagadnienia

Wymienione w niniejszej sekcji zagadnienia podsumowujące mają służyć do utrwalenia zdobytej wiedzy i umiejętności oraz do ułatwienia przygotowania się do sprawdzianu. Zagadnienia można opracować samodzielnie na podstawie dostępnych materiałów i doświadczeń zdobytych na laboratorium.

1. **Warstwa sprzętowa – teoria i praktyka.** Zagadnienia technologiczne i sprzętowe związane z tematyką ćwiczenia
 - a. Charakterystyka warstwy fizycznej interfejsu danych (obrazu) dla wyświetlacza LCD:
 - sygnały interfejsu SPI: MOSI/DIN, MISO/SDO, CLK, CS, DC, RST – do czego służą, jaka jest koncepcja ich zastosowania?
 - przesyłanie danych graficznych, sposób przesyłania danych, m.in.
 1. równoległy czy szeregowy?
 2. synchroniczny czy asynchroniczny?
2. **Część programowa – teoria i praktyka.** Rozumienie *co się dzieje* w programie. Znajomość poniższych przykładowych zagadnień na podstawie samodzielnej analizy kodu m.in. z udostępnionego projektu startowego, kodu napisanego w ramach realizacji ćwiczeń oraz dostępnej dokumentacji:
 - a. Gdzie (fizycznie w której pamięci) w projekcie programowym przy realizacji ćwiczenia przechowywane są dane stanowiące obraz wyświetlany na LCD?
 - b. Co robimy w funkcji inicjalizującej sterownik wyświetlacza?
 - c. Format piksela i kodowanie barw w ćwiczeniu:
3. **Wiedza ogólna na temat wyświetlania obrazu cyfrowego**
 - a. W jaki sposób kodowane są kolory w projekcie przy domyślnych ustawieniach i zestawie sterowników?
 - b. Bardziej ogólnie - umiejętność uzyskiwania zadanych barw i przezroczystości przy wskazanym kodowaniu formatu piksela w zakresie typowych kolorów: czerwony (R), zielony (G), niebieski (B), cyjan (C), magenta (M), żółty (Y), czarny oraz w skali szarości, w tym czarny i biały. Oczywiście nie ma konieczności rozpoznawania kolorów, należy przede wszystkim opanować (przynajmniej pamięciowo), z których składowych R, G, B i w jakich proporcjach utworzone są poszczególne kolory R, G, B, C, M, Y, czarny, biały i różne jasności koloru szarego. Nie musimy wiedzieć, jak wyglądają poszczególne kolory, a tym bardziej nie musimy także umieć ich rozpoznawać. Należy jednak wiedzieć, z jakich składają się komponentów R, G i B. Warto wiedzieć jakimi liczbami w systemie szesnastkowym zakodować kolory:
 - podstawowe RGB: czerwony, zielony, niebieski,
 - CMY: cyjan, fuksja (magenta), żółty,

- biały,
- czarny,
- odcienie kolorów o jasnościach: 25%, 50%, 75% (przyjmijmy, że 25% jest ciemniejszy niż 75%), np. szary o jasności 25% lub ciemnozielony o jasności 25%,